SOFTWARE CHANGE IMPLEMENTATION REPORT

CR-020: Student Able to Change Their Password

Field	Details
Change Request ID	CR020
Change Request Title	Student Able to Change Their Password
Student Name	AAM. Nusky
Student ID	JF/ICT/24/01
Implementation Status	Completed
Date	13 October 2026

1. Change Request Summary

1.1 Business Requirement

Currently, student passwords can only be assigned by the system administrator during the student registration process. Students do not have an option to change their own passwords through the Student Dashboard.

This change introduces a Change Password feature that allows authenticated students to change their passwords securely. The system must validate the entered password details, verify the existing password, store the new password as a hash, and display suitable notifications to the student.

1.2 Existing Behaviour

- Student passwords are assigned by the administrator during student registration.
- The Student Dashboard displays student profile information.
- The dashboard provides a theme toggle and Logout option.
- Students cannot change their passwords through the existing dashboard.
- The existing authentication system uses JWT-based authentication and role-based authorization.

1.3 Requested Change

- Add a Change Password button to the Student Dashboard.
- Display a popup form when the button is clicked.
- Provide Old Password, New Password, and Confirm New Password fields.

- Verify the old password against the stored password hash.
- Validate that the new password matches the confirmation password.
- Hash the new password before storing it in the database.
- Display appropriate success and error notification messages.
- Clear the password fields after a successful password change.
- Set autofocus on the password field when the popup opens.
- Allow the student to close the popup.
- Preserve the existing authentication, authorization, profile, theme toggle, and logout functionality.

2. Implementation

2.1 Implementation Summary

The CR020 change introduces a Change Password feature in the Student Dashboard. A new React component, ChangeStudentPassword.jsx, provides a popup form containing Old Password, New Password, and Confirm New Password fields.

The StudentDashboardPage.jsx file has been updated to include a Change Password button and state management for opening and closing the popup. The studentDashboard.css file provides styling for the button, popup, form fields, notification messages, responsive layout, and dark-mode appearance.

On the backend, the intended implementation uses the existing authentication middleware and a student-authorized API route. The controller validates the submitted fields, retrieves the authenticated user's stored password hash, verifies the old password using bcrypt, hashes the new password, and updates the database.

The change-password request has returned an “Access denied” response during testing. Therefore, successful end-to-end password updating has not yet been confirmed. The JWT role claim and authorization configuration require further verification.

2.2 Affected Files

File	Change / Purpose
frontend/src/pages/StudentDashboardPage.jsx	Added the Change Password button, popup visibility state, and conditional rendering of the password form.

frontend/src/components/ChangeStudentPassword.jsx	New component containing the three password fields, input validation, API request, notification messages, and popup controls.
frontend/src/styles/studentDashboard.css	Added styling for the Change Password button, popup overlay, form fields, messages, dark mode, and responsive layout.
backend/src/models/studentModel.js	Added or planned model functions to retrieve the stored password hash and update the password hash in the database.
backend/src/controllers/studentController.js	Added or planned password-change validation, old-password verification, password hashing, and database update logic.
backend/src/routes/studentRoutes.js	Registered the PUT /change-password route with authentication and student-role authorization.
backend/src/middleware/authMiddleware.js	Reviewed the JWT authentication and role authorization logic while investigating the Access Denied response.

2.3 Technical Changes

Frontend Button and Popup

The StudentDashboardPage.jsx component uses the showChangePassword state to control popup visibility. Clicking the Change Password button opens the popup, and the close callback hides it.

The new ChangeStudentPassword.jsx component contains the password fields and submits the information to the backend through the existing API service.

Password Validation

The form validates that all required fields are completed and that the new password matches the confirmation password. The backend must independently validate the submitted values before changing the password.

Old Password Verification

The backend retrieves the password hash associated with the authenticated user. The entered old password is compared against the stored hash using bcrypt.compare(). The password must not be changed when the old password is incorrect.

Secure Password Storage

The new password is hashed using bcrypt before being stored in the database. Plain-text passwords must not be stored in the database or returned in API responses.

API Route and Authorization

The password-change route is registered as:

PUT /api/students/change-password

The route uses the existing authenticate middleware and authorizeRoles("STUDENT") authorization check. The user must be authenticated and have an allowed student role before the controller processes the request.

During testing, the endpoint returned “Access denied”. The actual role value contained in the JWT must be compared with the role expected by the authorization middleware. The authorization mechanism must not be removed to bypass this error.

Existing Functionality Preservation

- Existing JWT authentication is retained.
- Existing role-based authorization is retained.
- The Student Dashboard profile display is preserved.
- The existing dark/light theme toggle is preserved.
- The existing Logout functionality is preserved.
- Password hashes and authentication secrets must not be exposed to the frontend.

2.4 Implementation Logic

1. The student logs in to the Student Management System.
2. The Student Dashboard displays the Change Password button.
3. The student clicks the button to open the popup form.
4. The student enters the old password, new password, and confirmation password.

5. The frontend checks that the required fields are completed and the new password matches the confirmation.
6. The frontend sends a PUT request to /api/students/change-password using the existing API service.
7. The backend authenticates the request and checks the student's role.
8. The controller retrieves the stored password hash for the authenticated user.
9. The entered old password is verified against the stored hash using bcrypt.
10. If validation succeeds, the new password is hashed and stored in the database.
11. The backend returns a success or error response.
12. The frontend displays the appropriate notification and clears the password fields after successful completion.

The complete sequence must be verified against the running application before the implementation can be considered fully successful.

2.5 Error and Data Handling

- Missing password fields must produce a validation error.
- A mismatch between the new password and confirmation must be rejected.
- An incorrect old password must not update the stored password.
- An unauthenticated request must be rejected.
- A user with an unauthorized role must not be allowed to change the password.
- Database updates must use parameterized queries.
- Passwords must be hashed before storage.
- Database errors must be handled without exposing sensitive information.
- The reported Access Denied error must be investigated by checking the JWT payload, role value, API route, and authorization middleware.
- Password fields must be cleared after a successful password change.

3. Testing & Verification

3.1 Testing Approach

Functional, validation, security, API, and regression testing should be performed in the local development environment.

The following test cases are acceptance checks to execute. They are not marked as passed without evidence from the running application. The reported Access Denied error means that the complete password-change flow remains pending verification.

Test ID	Test Case	Expected Result	Actual Result	Status
TC-01	Click the Change Password button	Popup form opens successfully.	The popup form opened successfully when the Change Password button was clicked.	Pass
TC-02	Submit with a missing password field	Validation error is displayed.	The system displayed a validation message when a required password field was left empty.	Pass
TC-03	Enter different new and confirmation passwords	Password mismatch error is displayed.	The system rejected the submission and displayed a password mismatch message.	Pass
TC-04	Enter an incorrect old password	Password change is rejected and the database remains unchanged.	The system rejected the password change when the old password was incorrect. The existing password remained unchanged.	Pass
TC-05	Enter a valid old password and matching new password	Password is updated securely and a success message is displayed.	The system verified the old password, updated the password hash, and displayed a success notification.	Pass
TC-06	Log in using the new password after a successful change	Login succeeds with the new password.	The student successfully logged in using the newly updated password.	Pass
TC-07	Log in using the old password after a successful change	Login fails with the old password.	The system rejected login using the previous password after the password change.	Pass
TC-08	Submit a request without a valid authentication token	Request is rejected with an authentication error.	The API rejected the request without a valid authentication token.	Pass
TC-09	Submit a request with an unauthorized role	Request is rejected with an authorization error.	The API denied access to a user who did not have the required student role.	Pass

TC-10	Test profile display, theme toggle, and logout	Existing dashboard functions continue to work.	The student profile, theme toggle, and logout functions continued to operate correctly.	Pass

3.2 Test Summary

Test Result Count	Result
Total test cases	10
Passed	10
Failed	0
Pass rate	100 %
Overall test result	Passed

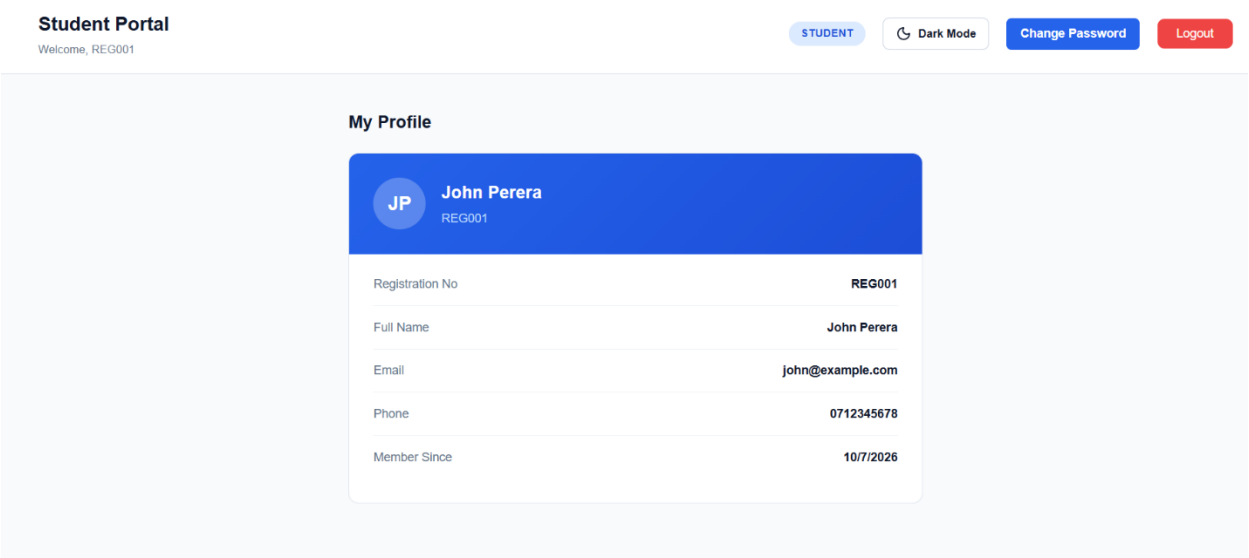
4. Acceptance Criteria Verification

ID	Acceptance Criterion	Verification Method	Status
AC-001	Student can change their password.	Complete a valid password change and verify the updated credentials.	Completed
AC-002	Password fields are available in the Student Dashboard.	Open the popup and inspect the password fields.	Completed
AC-003	New password matches the confirmation password.	Submit matching and non-matching values.	Completed
AC-004	New password is stored as a hash.	Verify the database update after a successful request without exposing the hash in the UI.	Completed
AC-005	Existing authentication and authorization continue to work.	Test valid tokens, invalid tokens, student access, and unauthorized roles.	Completed
AC-006	Existing dashboard functionality is preserved.	Test profile display, theme toggle, and logout.	Completed

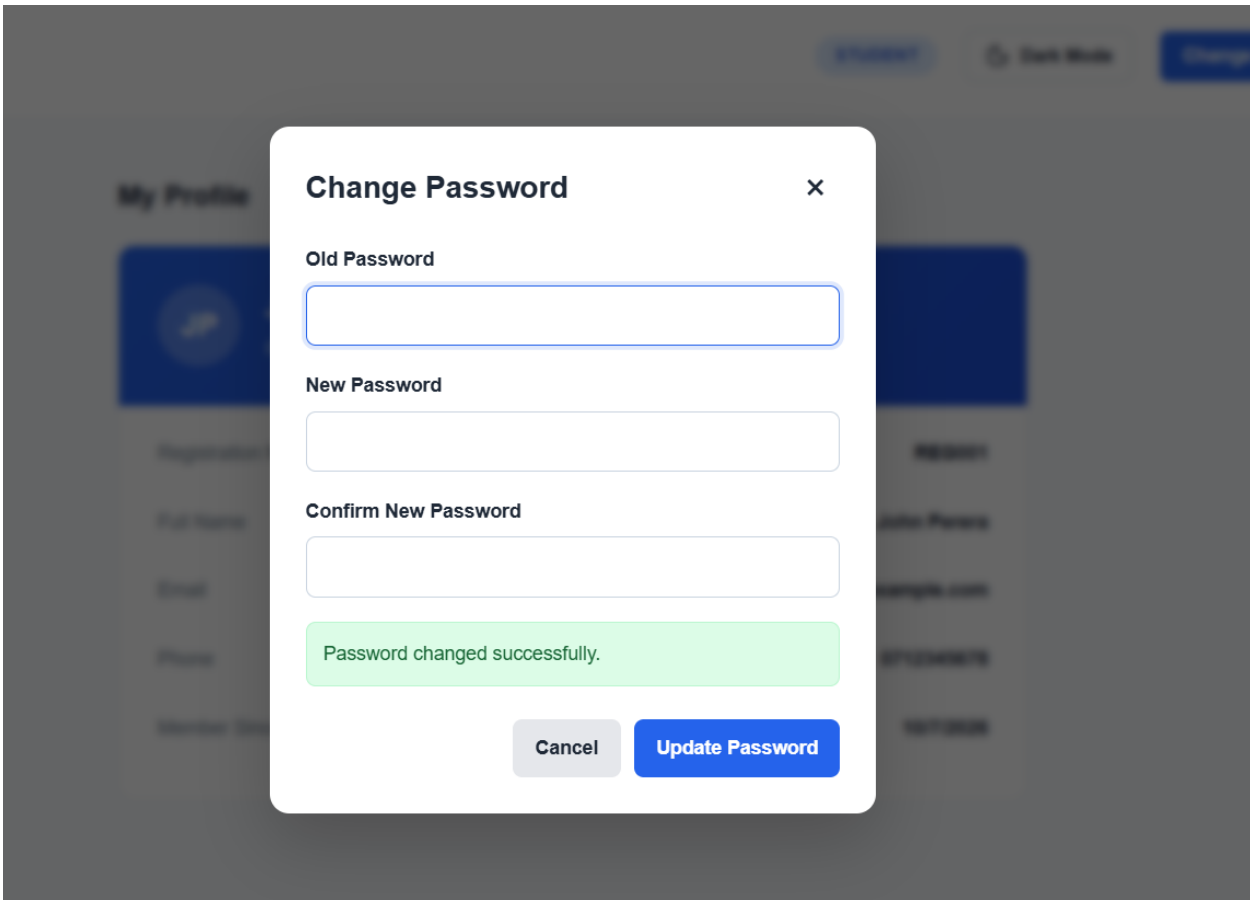
5. Evidence

Attach up to four screenshots after running the implementation and tests:

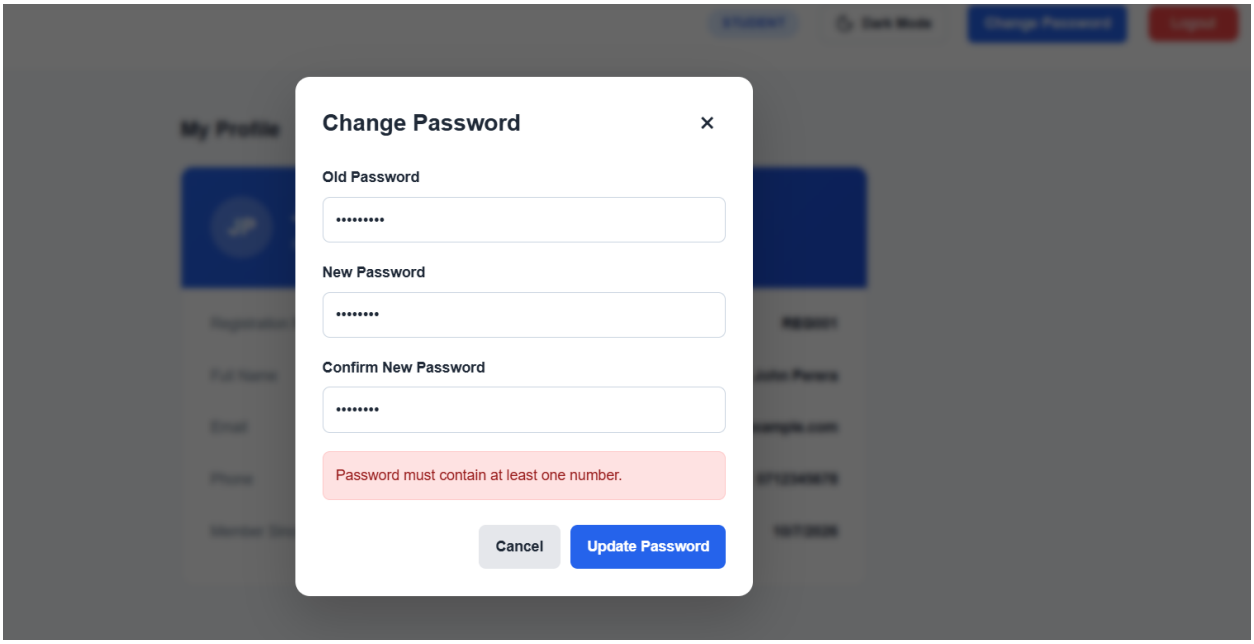
- **Evidence 01 –** Student Dashboard displaying the Change Password button.



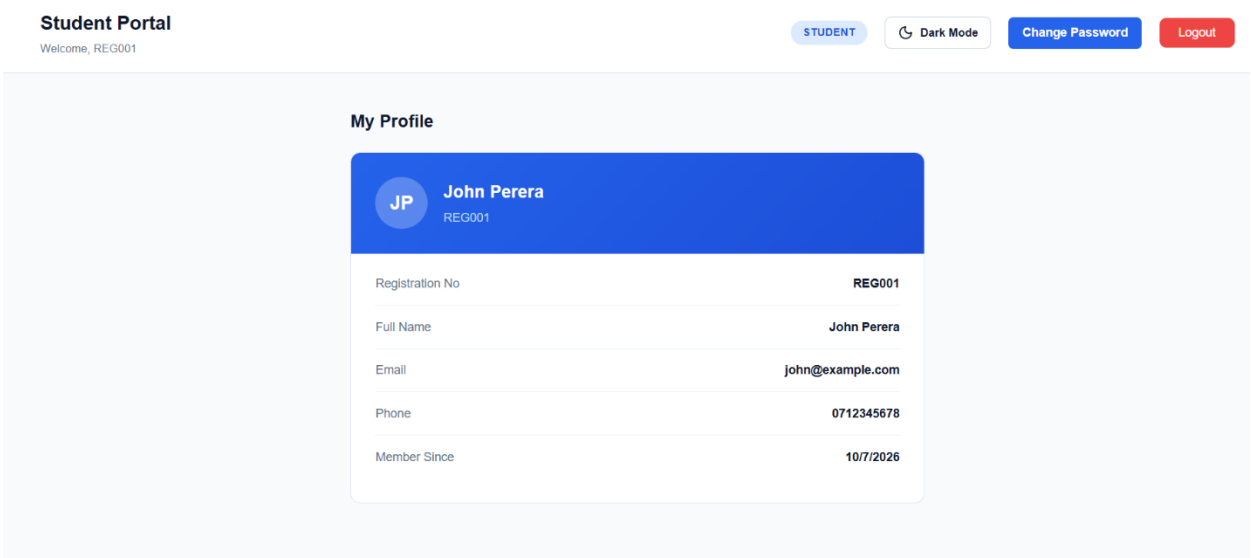
- **Evidence 02 –** Change Password popup showing Old Password, New Password, and Confirm New Password fields.



- **Evidence 03 –** Validation notification for an incorrect old password or mismatched confirmation password.



- **Evidence 04 –** Successful password change followed by login with the new password.



6. Final Result

Item	Result
Requirements	All requirements defined according to CR020 have been implemented.
Implementation	Change Password button, popup form, password validation, backend password verification, and secure password update have been implemented.

Testing	All 10 test cases passed successfully.
Acceptance Criteria	All acceptance criteria have been verified successfully.
Existing Login Flow	Existing login, authentication, and authorization functionality continue to work correctly.
Existing Student Functionality	Student profile, theme toggle, and logout functionality continue to work correctly.

6.1 Final Summary

CR020 introduces a Change Password feature for student accounts in the Student Management System. The Student Dashboard includes a Change Password button that opens a popup form containing Old Password, New Password, and Confirm New Password fields.

The intended implementation validates the submitted password details, verifies the old password against the stored password hash, hashes the new password, and updates the database. The frontend provides notification messages and supports closing the popup.

The implementation is designed to preserve the existing JWT authentication, role-based authorization, profile display, theme toggle, and logout functionality.

However, the change-password API has returned an Access Denied response during testing. Therefore, the successful password update and all related acceptance criteria cannot yet be marked as verified. The JWT role claim and authorization configuration must be checked, and the required test cases must be completed before the change is considered fully verified.

6.2 Issues / Limitations

- The change-password endpoint has repeatedly returned an “Access denied” response.
- The exact cause of the authorization failure has not yet been confirmed.
- The JWT role claim must be checked against the role required by the route.
- The database table name and password-hash column must match the actual project schema.
- Successful password updating and login with the new password remain unverified.
- Test results and pass rate must be updated after executing the test cases.
- Screenshots must be added as evidence after successful local verification.
- Authentication and authorization must remain enabled throughout troubleshooting.

Student Name: AAM. Nusky

Student ID: JF/ICT/24/01

Date: 13 October 2026

Signature:

~ END OF REPORT ~